

Labo 05 — Labo pratique : du JDK au JRE, de Node à nginx

Objectif : construire la même API deux fois — en une étape puis en multi-stage — mesurer la différence, faire de même pour un front « Angular », puis toucher aux caches de build et à une image sans shell.

Prérequis — Labo 04 terminé : `~/labo-docker/04/Api.java` existe, les images `eclipse-temurin:21-jdk` et `eclipse-temurin:21-jre-alpine` sont présentes.

Fichiers fournis (`files/`) - `web/package.json` et `web/src/index.html` — un faux projet Angular. Le « build » sera simulé par un `cp` : on reproduit la **forme** d'un projet front, pas son contenu.

Vous écrirez vous-même chaque Dockerfile : c'est l'exercice.

Étape 1 — L'image « tout-en-un »

```
mkdir -p ~/labo-docker/05 && cd ~/labo-docker/05
cp ~/labo-docker/04/Api.java .
```

Créez `Dockerfile.mono` — compilation **et** exécution dans la même image :

```
FROM docker.io/library/eclipse-temurin:21-jdk
WORKDIR /app
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -f Dockerfile.mono -t api-mono:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-
mono|temurin'
```

Observez `localhost/api-mono 1.0 488 MB` — exactement la taille de `eclipse-temurin:21-jdk`. Le JAR de 2 Ko n'a rien ajouté ; le JDK a tout apporté.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version'
```

Observez `Api.java`, `api.jar`, `build`, et `javac 21.0.x` : le code source **et** le compilateur sont dans l'image de production.

Explication. Cette image fonctionne parfaitement. C'est ce qui la rend dangereuse : rien ne signale que 480 Mo d'outillage et vos sources partent avec chaque déploiement.

Étape 2 — Le multi-stage

Créez `Dockerfile` :

```
# ----- stage 1 : build -----
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN mkdir -p build && javac -d build Api.java && jar --create --file api.jar --main-class Api -C build .

# ----- stage 2 : runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /src/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

```
podman build -t api-multi:1.0 .
```

Observez les préfixes [1/2] STEP 1/4 ... puis [2/2] STEP 1/6 ... : Buildah numérote les stages, et seul le dernier se termine par COMMIT api-multi:1.0 .

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-mono|api-multi'
podman run --rm --entrypoint sh api-multi:1.0 -c 'ls /app; javac -version'
podman run --rm --entrypoint ls api-multi:1.0 /src
```

Observez 209 MB contre 488 MB , puis api.jar seul, sh: javac: not found , et ls: cannot access '/src': No such file or directory .

Explication. Le stage build a existé le temps de la compilation, puis a été jeté. L'image finale n'en connaît que le fichier copié par COPY --from . Ni sources, ni JDK, ni dossier /src : ils n'ont jamais fait partie de ses couches.

```
podman history api-multi:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6
```

Observez une couche de 4.61kB pour le COPY du JAR : tout le reste vient de l'image de base.

PIÈGE

COPY --from=build /src /app aurait recopié le dossier entier, Api.java et build/ compris. On copie l'**artefact**, pas le répertoire de travail. C'est la question 3.

Étape 3 — Regarder à l'intérieur d'un stage

Un stage jeté n'est pas inspectable... sauf si on demande à s'arrêter là :

```
podman build --target build -t api-build-stage .
podman run --rm api-build-stage ls -la /src
```

Observez Api.java , api.jar , build/ : c'est le contenu exact du stage au moment où le stage 2 y a copié api.jar .

Explication. `--target` est l'outil de diagnostic du multi-stage : quand un `COPY --from` échoue avec `no such file or directory`, on construit le stage seul et on regarde, au lieu de deviner les chemins (question 12).

```
podman rmi api-build-stage
```

Étape 4 — Le front : Node construit, nginx sert

```
mkdir -p web && cp -r <chemin-du-labo>/files/web/* web/  
ls -R web
```

Créez `web/Dockerfile` :

```
FROM docker.io/library/node:22-alpine AS build  
WORKDIR /app  
COPY package.json .  
RUN echo "npm ci (simule)"  
COPY src ./src  
RUN mkdir -p dist/web/browser && cp src/index.html dist/web/browser/ && echo "ng build  
(simule)"  
  
FROM docker.io/library/nginx:alpine  
COPY --from=build /app/dist/web/browser /usr/share/nginx/html  
EXPOSE 80
```

```
podman build -t web-multi:1.0 web  
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'web-  
multi|node|nginx'
```

Observez `web-multi 1.0 64.2 MB` — la taille de `nginx:alpine`, exactement — alors que `node:22-alpine` pèse `167 MB`.

```
podman run -d --name web -p 18081:80 web-multi:1.0  
curl -s localhost:18081/  
podman rm -f -t 0 web
```

Observez votre `index.html` servi par nginx. Ouvrez aussi `http://localhost:18081/` dans le navigateur Windows.

Explication. Node a servi à « construire » (ici, un `cp` tient lieu de `ng build`), puis a disparu. Le front en production est un serveur de fichiers statiques : c'est pour cela que l'image `web` d'une stack Angular ne contient jamais Node.

→ ANGULAR

Sur un vrai projet, remplacez `RUN echo "npm ci (simule)"` par `RUN npm ci` et le `cp` par `RUN npm run build`, et copiez `dist/<nom-du-projet>/browser`. La séparation `COPY package*.json → npm ci → COPY . .` est celle du cache du labo 04 : les 900 Mo de `node_modules` ne sont retéléchargés que si `package-lock.json` change.

Étape 5 — Un cache qui survit aux builds

Créez `Dockerfile.cache` :

```
FROM docker.io/library/eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY Api.java .
RUN --mount=type=cache,target=/root/.m2 sh -c 'echo "dep-$(date +%s)" >> /root/.m2/marker; cat
/root/.m2/marker' \
&& mkdir -p build && javac -d build Api.java

FROM docker.io/library/alpine
COPY --from=build /src/build /app
```

Le `marker` simule le dépôt Maven `~/.m2` : chaque build y ajoute une ligne.

```
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
podman build --no-cache -f Dockerfile.cache -t cache-demo . 2>&1 | grep dep-
```

Observez une ligne `dep-...` au premier build, **deux** au second — alors que `--no-cache` a tout reconstruit. Le dossier `/root/.m2` a survécu entre les deux builds.

```
podman run --rm cache-demo ls /root/.m2
```

Observez `ls: /root/.m2: No such file or directory` : le cache n'est **pas** dans l'image.

Explication. Le *cache mount* est un dossier tenu par Buildah dans votre stockage utilisateur, monté dans le conteneur de build le temps d'une instruction. Sur un vrai projet, `RUN --mount=type=cache,target=/root/.m2 mvn package` évite de retélécharger 300 Mo de dépendances à chaque build — même quand `pom.xml` change. Aucune ligne `# syntax=` n'a été nécessaire : Buildah comprend `--mount` nativement.

Étape 6 — musl ou glibc ?

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --
version 2>&1 | head -1; head -1 /etc/os-release'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version 2>&1
| head -1; head -1 /etc/os-release'
```

Observez `musl libc (x86_64)` / `Alpine Linux` d'un côté, `ldd (Ubuntu GLIBC 2.xx)` / `Ubuntu` de l'autre.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'apk info |
wc -l'
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'dpkg -l | grep -c
^ii'
```

Observez environ `73` paquets contre `140`.

Explication. C'est la commande à lancer **avant** de migrer une image vers Alpine : si une bibliothèque native de votre application a été compilée pour `glibc`, elle ne se chargera pas avec `musl`. Le nombre de paquets, lui, est ce que compte un scanner de vulnérabilités : moitié moins de paquets, moitié moins de CVE potentielles.

Étape 7 — Distroless : pas de shell du tout

```
podman pull gcr.io/distroless/java21-debian12
podman images --format '{{.Repository}} {{.Size}}' | grep distroless
```

Observez `194 MB` — moins que le JRE Alpine, alors que c'est du Debian.

Créez `Dockerfile.distroless` :

```
FROM gcr.io/distroless/java21-debian12
COPY --from=localhost/api-multi:1.0 /app/api.jar /app/api.jar
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

```
podman build -f Dockerfile.distroless -t api-distroless:1.0 .
podman run -d --name d -p 18082:8080 api-distroless:1.0
sleep 2 ; curl -s localhost:18082/actuator/health ; echo
podman exec d sh -c 'ls'
podman exec d id
podman rm -f -t 0 d
```

Observez `{"status":"UP"}` — l'API tourne — puis `executable file sh not found in $PATH` et la même erreur pour `id` : il n'y a **rien** dans cette image à part Java et votre JAR.

Explication. `COPY --from=` accepte aussi le nom d'une **image** existante, pas seulement un stage. Et une image sans shell est une image où un attaquant qui obtient l'exécution de code ne trouve ni `sh`, ni `curl`, ni `apt` — mais où vous non plus ne pouvez pas entrer. On compense par des logs riches, `/actuator`, et `podman cp` pour extraire un fichier.

Nettoyage

```
podman rmi api-mono:1.0 api-multi:1.0 web-multi:1.0 cache-demo api-distroless:1.0 \
    gcr.io/distroless/java21-debian12 docker.io/library/node:22-alpine 2>/dev/null
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}'
```

Observez qu'il reste `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre` et `eclipse-temurin:21-jre-alpine`. Gardez `~/labo-docker/05/Dockerfile` : la stack des labos suivants s'en servira.

Ce que vous devez pouvoir affirmer maintenant

- Une image mono-stage pèse le poids de son outillage : 488 Mo pour un JAR de 2 Ko.

- Le multi-stage ne garde que l'artefact copié : 209 Mo, sans sources ni compilateur — et `--target` permet d'inspecter un stage.
- Le front Angular en production est une image nginx de 64 Mo ; Node n'y est pas.
- Un *cache mount* survit aux builds et n'entre pas dans l'image ; Buildah le gère sans `#syntax=`.
- Alpine = `musl`, Ubuntu = `glibc` : `ldd --version` le dit, et un test le valide.
- Distroless : `{"status": "UP"}` mais pas de `sh` — sécurité contre observabilité.